REVISA — Pacote de Implantação do Backend

1. Objetivo

Este pacote fecha os últimos elementos necessários para tirar o backend da condição de blueprint e levá-lo a um estado executável de implantação local e base de homologação.

Escopo consolidado:

- testes de integração mínimos por domínio
- `get_current_user` real com JWT e RBAC
- dependencies/decorators de permissão e escopo
- auditoria automática centralizada
- bootstrap consolidado do backend para rodar sem stubs

A meta deste pacote é permitir que a equipe materialize os arquivos e tenha um backend capaz de:

- subir com FastAPI
- autenticar usuários reais
- aplicar RBAC básico
- validar escopo mínimo
- persistir operações centrais
- gerar auditoria automática em mutações
- rodar testes mínimos de integração

---

2. Estrutura final adicional do backend

```
apps/api/
├── app/
│   ├── main.py
│   ├── core/
│   │   ├── settings.py
│   │   ├── database.py
│   │   ├── security.py
│   │   ├── auth.py
│   │   ├── permissions.py
│   │   ├── scope.py
│   │   ├── audit.py
│   │   └── startup.py
│   ├── api/
│   │   ├── deps/
│   │   │   ├── auth.py
│   │   │   ├── permissions.py
│   │   │   └── scope.py
│   │   └── v1/
│   │       ├── api.py
```

```
│   │       └── routers/
│   ├── domain/
│   │   ├── iam/
│   │   ├── core/
│   │   ├── territory/
│   │   ├── polo/
│   │   ├── workflow/
│   │   ├── analytics/
│   │   └── governance/
│   ├── shared/
│   │   ├── audit.py
│   │   ├── responses.py
│   │   └── types.py
│   └── tests/
│       ├── conftest.py
│       ├── integration/
│       │   ├── test_auth_login.py
│       │   ├── test_persons.py
│       │   ├── test_contacts_capture.py
│       │   ├── test_polos.py
│       │   ├── test_tasks.py
│       │   └── test_dashboards.py
│       └── fixtures/
└── scripts/
    ├── seed_initial_data.py
    └── bootstrap_admin.py
```

---

3. JWT real, RBAC e usuário autenticado

3.1 `apps/api/app/core/security.py`

```python
from datetime import datetime, timedelta, timezone
from typing import Any

import jwt
from passlib.context import CryptContext

from app.core.settings import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")


def hash_password(password: str) -> str:
    return pwd_context.hash(password)
```

```
def verify_password(password: str, password_hash: str) -> bool:
    return pwd_context.verify(password, password_hash)


def _encode_token(subject: str, secret: str, expires_delta: timedelta,
token_type: str) -> str:
    now = datetime.now(timezone.utc)
    payload: dict[str, Any] = {
        "sub": subject,
        "type": token_type,
        "iat": now,
        "exp": now + expires_delta,
    }
    return jwt.encode(payload, secret, algorithm="HS256")


def create_access_token(subject: str) -> str:
    return _encode_token(subject, settings.jwt_secret_key,
timedelta(minutes=30), "access")


def create_refresh_token(subject: str) -> str:
    return _encode_token(subject, settings.jwt_refresh_secret_key,
timedelta(days=7), "refresh")


def decode_access_token(token: str) -> dict[str, Any]:
    return jwt.decode(token, settings.jwt_secret_key, algorithms=["HS256"])


def decode_refresh_token(token: str) -> dict[str, Any]:
    return jwt.decode(token, settings.jwt_refresh_secret_key,
algorithms=["HS256"])
```

3.2 `apps/api/app/domain/iam/models.py`

```
from datetime import datetime
from uuid import uuid4

from sqlalchemy import Boolean, DateTime, ForeignKey, String, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base


class User(Base):
    __tablename__ = "users"
    __table_args__ = {"schema": "iam"}
```

```
    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    person_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
nullable=True)
    username: Mapped[str] = mapped_column(String(120), unique=True,
nullable=False)
    email: Mapped[str] = mapped_column(String(180), unique=True,
nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="ACTIVE",
nullable=False)
    last_login_at: Mapped[datetime | None] = mapped_column(DateTime)
    must_reset_password: Mapped[bool] = mapped_column(Boolean,
default=False, nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class Role(Base):
    __tablename__ = "roles"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    code: Mapped[str] = mapped_column(String(80), unique=True,
nullable=False)
    name: Mapped[str] = mapped_column(String(120), nullable=False)


class Permission(Base):
    __tablename__ = "permissions"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    code: Mapped[str] = mapped_column(String(120), unique=True,
nullable=False)
    name: Mapped[str] = mapped_column(String(120), nullable=False)


class UserRole(Base):
    __tablename__ = "user_roles"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False)
    role_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
```

```
ForeignKey("iam.roles.id", ondelete="CASCADE"), nullable=False)
    is_primary: Mapped[bool] = mapped_column(Boolean, default=False,
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class RolePermission(Base):
    __tablename__ = "role_permissions"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    role_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.roles.id", ondelete="CASCADE"), nullable=False)
    permission_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.permissions.id", ondelete="CASCADE"), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class UserScopeAssignment(Base):
    __tablename__ = "user_scope_assignments"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False)
    scope_type: Mapped[str] = mapped_column(String(30), nullable=False)
    scope_ref_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)


class RefreshToken(Base):
    __tablename__ = "refresh_tokens"
    __table_args__ = {"schema": "iam"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id", ondelete="CASCADE"), nullable=False)
    token_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    expires_at: Mapped[datetime] = mapped_column(DateTime, nullable=False)
    revoked_at: Mapped[datetime | None] = mapped_column(DateTime)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

3.3 `apps/api/app/domain/iam/repository.py`

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.iam.models import Permission, RefreshToken, Role,
RolePermission, User, UserRole, UserScopeAssignment


class IAMRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_user_by_username(self, username: str) -> User | None:
        stmt = select(User).where(User.username == username)
        return self.db.execute(stmt).scalars().first()

    def get_user_by_id(self, user_id: str) -> User | None:
        stmt = select(User).where(User.id == user_id)
        return self.db.execute(stmt).scalars().first()

    def get_role_codes(self, user_id: str) -> list[str]:
        stmt = (
            select(Role.code)
            .join(UserRole, UserRole.role_id == Role.id)
            .where(UserRole.user_id == user_id)
        )
        return list(self.db.execute(stmt).scalars().all())

    def get_permission_codes(self, user_id: str) -> list[str]:
        stmt = (
            select(Permission.code)
            .join(RolePermission, RolePermission.permission_id ==
Permission.id)
            .join(Role, Role.id == RolePermission.role_id)
            .join(UserRole, UserRole.role_id == Role.id)
            .where(UserRole.user_id == user_id)
        )
        return list(set(self.db.execute(stmt).scalars().all()))

    def get_scopes(self, user_id: str) -> list[UserScopeAssignment]:
        stmt = select(UserScopeAssignment).where(UserScopeAssignment.user_id
== user_id)
        return list(self.db.execute(stmt).scalars().all())

    def save_refresh_token(self, entity: RefreshToken) -> RefreshToken:
        self.db.add(entity)
        self.db.flush()
```

```
        self.db.refresh(entity)
        return entity
```

3.4 `apps/api/app/core/auth.py`

```
from dataclasses import dataclass
from hashlib import sha256
from typing import Iterable

from app.core.security import create_access_token, create_refresh_token,
decode_access_token, verify_password
from app.domain.iam.models import RefreshToken
from app.domain.iam.repository import IAMRepository


@dataclass
class CurrentUser:
    id: str
    username: str
    email: str
    roles: set[str]
    permissions: set[str]
    scope_map: dict[str, set[str]]
    is_global_admin: bool

    @property
    def allowed_polo_ids(self) -> set[str]:
        return self.scope_map.get("POLO", set())

    @property
    def allowed_gabinete_ids(self) -> set[str]:
        return self.scope_map.get("GABINETE", set())

    @property
    def allowed_vereador_ids(self) -> set[str]:
        return self.scope_map.get("VEREADOR", set())


class AuthService:
    def __init__(self, repo: IAMRepository):
        self.repo = repo

    def login(self, username: str, password: str):
        user = self.repo.get_user_by_username(username)
        if not user or user.status != "ACTIVE":
            return None
        if not verify_password(password, user.password_hash):
            return None

        access_token = create_access_token(str(user.id))
```

```
        refresh_token = create_refresh_token(str(user.id))
        refresh_hash = sha256(refresh_token.encode()).hexdigest()
        self.repo.save_refresh_token(
            RefreshToken(
                user_id=user.id,
                token_hash=refresh_hash,
                expires_at=decode_access_token(access_token)["exp"],
            )
        )
        return {
            "access_token": access_token,
            "refresh_token": refresh_token,
            "token_type": "Bearer",
            "expires_in": 1800,
        }

    def build_current_user(self, access_token: str) -> CurrentUser:
        payload = decode_access_token(access_token)
        user_id = payload["sub"]
        user = self.repo.get_user_by_id(user_id)
        if not user:
            raise ValueError("User not found")

        roles = set(self.repo.get_role_codes(user_id))
        permissions = set(self.repo.get_permission_codes(user_id))
        scopes = self.repo.get_scopes(user_id)
        scope_map: dict[str, set[str]] = {}
        for scope in scopes:
            scope_map.setdefault(scope.scope_type,
set()).add(str(scope.scope_ref_id))

        return CurrentUser(
            id=str(user.id),
            username=user.username,
            email=user.email,
            roles=roles,
            permissions=permissions,
            scope_map=scope_map,
            is_global_admin="ADM_GERAL_REVISA" in roles,
        )
```

3.5 `apps/api/app/api/deps/auth.py`

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.auth import AuthService
from app.core.database import get_db
from app.domain.iam.repository import IAMRepository
```

```
bearer_scheme = HTTPBearer(auto_error=True)


def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db),
):
    try:
        service = AuthService(IAMRepository(db))
        return service.build_current_user(credentials.credentials)
    except Exception:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid or expired token")
```

---

4. Dependencies de permissão e escopo

4.1 `apps/api/app/core/permissions.py`

```
def has_permission(current_user, permission_code: str) -> bool:
    return current_user.is_global_admin or permission_code in
current_user.permissions
```

4.2 `apps/api/app/core/scope.py`

```
def in_scope(current_user, scope_type: str, scope_ref_id: str | None) ->
bool:
    if current_user.is_global_admin:
        return True
    if scope_ref_id is None:
        return True
    return scope_ref_id in current_user.scope_map.get(scope_type, set())
```

4.3 `apps/api/app/api/deps/permissions.py`

```
from fastapi import Depends, HTTPException, status

from app.api.deps.auth import get_current_user
from app.core.permissions import has_permission


def require_permission(permission_code: str):
    def dependency(current_user = Depends(get_current_user)):
        if not has_permission(current_user, permission_code):
```

```
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Missing permission: {permission_code}",
            )
        return current_user
    return dependency
```

4.4 `apps/api/app/api/deps/scope.py`

```
from fastapi import HTTPException, status

from app.core.scope import in_scope


def ensure_scope(current_user, scope_type: str, scope_ref_id: str | None):
    if not in_scope(current_user, scope_type, scope_ref_id):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail=f"Out of scope: {scope_type}",
        )
```

4.5 Uso canônico no router

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from uuid import UUID

from app.api.deps.permissions import require_permission
from app.api.deps.scope import ensure_scope
from app.core.database import get_db

router = APIRouter()


@router.get("/{id}/beneficiarios")
def list_beneficiarios(
    id: UUID,
    current_user = Depends(require_permission("polo.manage_beneficiary")),
    db: Session = Depends(get_db),
):
    ensure_scope(current_user, "POLO", str(id))
    return []
```

---

5. Auditoria automática centralizada

5.1 `apps/api/app/core/audit.py`

```
import json
from functools import wraps

from sqlalchemy.orm import Session

from app.shared.audit import write_audit_log


def audited_mutation(action: str, entity_schema: str, entity_name: str):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)

            db: Session | None = kwargs.get("db")
            current_user = kwargs.get("current_user")
            if db is None:
                for arg in args:
                    if isinstance(arg, Session):
                        db = arg
                        break
            if current_user is None:
                current_user = kwargs.get("user")

            entity_id = getattr(result, "id", None)
            if db is not None:
                write_audit_log(
                    db,
                    user_id=getattr(current_user, "id", None),
                    action=action,
                    entity_schema=entity_schema,
                    entity_name=entity_name,
                    entity_id=entity_id,
                    old_values_json=None,
                    new_values_json=json.dumps({"id": str(entity_id)}) if
entity_id else None,
                )
            return result
        return wrapper
    return decorator
```

5.2 Uso canônico em service

```
from app.core.audit import audited_mutation


class CoreService:
    def __init__(self, repo):
        self.repo = repo

    @audited_mutation(action="CREATE", entity_schema="core",
entity_name="persons")
    def create_person(self, payload, db=None, current_user=None):
        entity = Person(**payload.model_dump())
        return self.repo.create_person(entity)
```

5.3 Recomendação de extensão

Depois da materialização inicial, a evolução correta é passar a auditar:

- `CREATE`
- `UPDATE`
- `DELETE lógico`
- `EXPORT`
- `CONSENT_REVOKE`
- `ROLE_ASSIGN`
- `SCOPE_ASSIGN`

---

6. Bootstrap consolidado do backend

6.1 `apps/api/app/core/startup.py`

```
from fastapi.middleware.cors import CORSMiddleware


def configure_middlewares(app):
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
```

6.2 apps/api/app/api/v1/api.py

```
from fastapi import APIRouter

from app.api.v1.routers import auth, contacts_capture, dashboards, persons,
polos, tasks, users

api_router = APIRouter()
api_router.include_router(auth.router, prefix="/auth", tags=["Auth"])
api_router.include_router(users.router, prefix="/users", tags=["Users"])
api_router.include_router(persons.router, prefix="/persons",
tags=["Persons"])
api_router.include_router(contacts_capture.router, prefix="/contacts-
capture", tags=["ContactsCapture"])
api_router.include_router(polos.router, prefix="/polos", tags=["Polos"])
api_router.include_router(tasks.router, prefix="/tasks", tags=["Tasks"])
api_router.include_router(dashboards.router, prefix="", tags=["Dashboards"])
```

6.3 apps/api/app/api/v1/routers/auth.py

```
from fastapi import APIRouter, Depends, HTTPException, status
from pydantic import BaseModel
from sqlalchemy.orm import Session

from app.api.deps.auth import get_current_user
from app.core.database import get_db
from app.core.auth import AuthService
from app.domain.iam.repository import IAMRepository

router = APIRouter()


class LoginRequest(BaseModel):
    username: str
    password: str


@router.post("/login")
def login(payload: LoginRequest, db: Session = Depends(get_db)):
    result = AuthService(IAMRepository(db)).login(payload.username,
payload.password)
    if not result:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid credentials")
    db.commit()
    return result
```

```
@router.get("/me")
def me(current_user = Depends(get_current_user)):
    return {
        "id": current_user.id,
        "username": current_user.username,
        "email": current_user.email,
        "roles": sorted(list(current_user.roles)),
        "permissions": sorted(list(current_user.permissions)),
        "scopes": {k: sorted(list(v)) for k, v in
current_user.scope_map.items()},
    }
```

6.4 `apps/api/app/main.py`

```
from fastapi import FastAPI

from app.api.v1.api import api_router
from app.core.startup import configure_middlewares

app = FastAPI(title="REVISA Platform API", version="1.0.0")
configure_middlewares(app)
app.include_router(api_router, prefix="/api/v1")


@app.get("/health")
def health():
    return {"status": "ok"}
```

---

7. Ajustes canônicos dos routers existentes

7.1 `apps/api/app/api/v1/routers/persons.py`

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps.auth import get_current_user
from app.api.deps.permissions import require_permission
from app.core.database import get_db
from app.domain.core.repository import CoreRepository
from app.domain.core.schemas import PersonCreate, PersonOut
from app.domain.core.service import CoreService

router = APIRouter()


@router.get("", response_model=list[PersonOut])
```

```
def list_persons(
    current_user = Depends(require_permission("person.read")),
    db: Session = Depends(get_db),
):
    return CoreService(CoreRepository(db)).list_persons()


@router.post("", response_model=PersonOut, status_code=201)
def create_person(
    payload: PersonCreate,
    current_user = Depends(require_permission("person.create")),
    db: Session = Depends(get_db),
):
    result = CoreService(CoreRepository(db)).create_person(payload, db=db,
current_user=current_user)
    db.commit()
    return result
```

7.2 apps/api/app/domain/core/service.py

```
from app.core.audit import audited_mutation
from app.domain.core.models import Person
from app.domain.core.repository import CoreRepository
from app.domain.core.schemas import PersonCreate


class CoreService:
    def __init__(self, repo: CoreRepository):
        self.repo = repo

    def list_persons(self):
        return self.repo.list_persons()

    @audited_mutation(action="CREATE", entity_schema="core",
entity_name="persons")
    def create_person(self, payload: PersonCreate, db=None,
current_user=None):
        entity = Person(**payload.model_dump())
        return self.repo.create_person(entity)
```

7.3 apps/api/app/api/v1/routers/contacts_capture.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps.permissions import require_permission
from app.core.database import get_db
from app.domain.territory.repository import TerritoryRepository
```

```python
from app.domain.territory.schemas import ContactCaptureCreate,
ContactCaptureOut
from app.domain.territory.service import TerritoryService

router = APIRouter()


@router.get("", response_model=list[ContactCaptureOut])
def list_captures(
    current_user = Depends(require_permission("capture.read")),
    db: Session = Depends(get_db),
):
    return TerritoryService(TerritoryRepository(db)).list_captures()


@router.post("", response_model=ContactCaptureOut, status_code=201)
def create_capture(
    payload: ContactCaptureCreate,
    current_user = Depends(require_permission("capture.create")),
    db: Session = Depends(get_db),
):
    result = TerritoryService(TerritoryRepository(db)).create_capture(
        captured_by_user_id=current_user.id,
        payload=payload,
    )
    db.commit()
    return result
```

7.4 `apps/api/app/api/v1/routers/tasks.py`

```python
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.api.deps.permissions import require_permission
from app.core.database import get_db
from app.domain.workflow.repository import WorkflowRepository
from app.domain.workflow.schemas import TaskCreate, TaskOut
from app.domain.workflow.service import WorkflowService

router = APIRouter()


@router.get("", response_model=list[TaskOut])
def list_tasks(
    current_user = Depends(require_permission("task.read")),
    db: Session = Depends(get_db),
):
    return WorkflowService(WorkflowRepository(db)).list_tasks()
```

```
@router.post("", response_model=TaskOut, status_code=201)
def create_task(
    payload: TaskCreate,
    current_user = Depends(require_permission("task.create")),
    db: Session = Depends(get_db),
):
    result = WorkflowService(WorkflowRepository(db)).create_task(
        created_by_user_id=current_user.id,
        payload=payload,
    )
    db.commit()
    return result
```

---

8. Testes de integração mínimos por domínio

8.1 `apps/api/app/tests/conftest.py`

```
import os
import uuid

import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from app.core.database import Base, get_db
from app.main import app

TEST_DATABASE_URL = os.getenv("TEST_DATABASE_URL",
"sqlite+pysqlite:///:memory:")

engine = create_engine(TEST_DATABASE_URL, future=True)
TestingSessionLocal = sessionmaker(bind=engine, autoflush=False,
autocommit=False, future=True)


@pytest.fixture(scope="session", autouse=True)
def create_test_db():
    Base.metadata.create_all(bind=engine)
    yield
    Base.metadata.drop_all(bind=engine)


@pytest.fixture()
def db_session():
    connection = engine.connect()
    transaction = connection.begin()
```

```
    session = TestingSessionLocal(bind=connection)
    yield session
    session.close()
    transaction.rollback()
    connection.close()


@pytest.fixture()
def client(db_session):
    def override_get_db():
        try:
            yield db_session
        finally:
            pass

    app.dependency_overrides[get_db] = override_get_db
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()
```

8.2 apps/api/app/tests/integration/test_auth_login.py

```
from app.core.security import hash_password
from app.domain.iam.models import User


def test_login_success(client, db_session):
    user = User(
        username="admin",
        email="admin@revisa.local",
        password_hash=hash_password("123456"),
        status="ACTIVE",
    )
    db_session.add(user)
    db_session.commit()

    response = client.post(
        "/api/v1/auth/login",
        json={"username": "admin", "password": "123456"},
    )

    assert response.status_code == 200
    body = response.json()
    assert "access_token" in body
    assert body["token_type"] == "Bearer"
```

8.3 apps/api/app/tests/integration/test_persons.py

```
def test_list_persons_requires_auth(client):
    response = client.get("/api/v1/persons")
    assert response.status_code in (401, 403)
```

8.4 apps/api/app/tests/integration/test_contacts_capture.py

```
def test_create_capture_requires_auth(client):
    response = client.post(
        "/api/v1/contacts-capture",
        json={
            "origin": "MOBILE",
            "classification": "CIDADAO",
            "full_name": "Maria da Silva",
        },
    )
    assert response.status_code in (401, 403)
```

8.5 apps/api/app/tests/integration/test_polos.py

```
def test_list_beneficiarios_requires_auth(client):
    response = client.get("/api/v1/polos/
11111111-1111-1111-1111-111111111111/beneficiarios")
    assert response.status_code in (401, 403)
```

8.6 apps/api/app/tests/integration/test_tasks.py

```
def test_list_tasks_requires_auth(client):
    response = client.get("/api/v1/tasks")
    assert response.status_code in (401, 403)
```

8.7 apps/api/app/tests/integration/test_dashboards.py

```
def test_dashboard_requires_auth(client):
    response = client.get("/api/v1/vereadores/
11111111-1111-1111-1111-111111111111/dashboard")
    assert response.status_code in (401, 403)
```

Observação importante sobre os testes

Para SQLite em memória, schemas PostgreSQL e materialized views não serão plenamente compatíveis. Na prática, a melhor abordagem de homologação é usar PostgreSQL de teste em container. O `conftest.py` acima serve como base mínima, mas a versão correta para pipeline deve ser ajustada para PostgreSQL real.

---

9. Bootstrap administrativo inicial

9.1 `apps/api/scripts/bootstrap_admin.py`

```
import uuid

from app.core.database import SessionLocal
from app.core.security import hash_password
from app.domain.iam.models import User


def main():
    db = SessionLocal()
    try:
        exists = db.query(User).filter(User.username == "admin").first()
        if exists:
            print("admin já existe")
            return

        user = User(
            id=uuid.uuid4(),
            username="admin",
            email="admin@revisa.local",
            password_hash=hash_password("Admin@123"),
            status="ACTIVE",
        )
        db.add(user)
        db.commit()
        print("admin criado")
    finally:
        db.close()


if __name__ == "__main__":
    main()
```

---

10. Ordem de materialização recomendada

Etapa 1

- materializar `core/security.py`, `core/auth.py`, `api/deps/auth.py`, `api/deps/permissions.py`, `api/deps/scope.py`
- materializar `domain/iam/models.py` e `domain/iam/repository.py`
- trocar routers para `require_permission`

Etapa 2

- materializar `core/audit.py` e `shared/audit.py`
- aplicar `@audited_mutation` nos serviços de criação principais

Etapa 3

- materializar `main.py`, `startup.py`, `auth.py`, `api.py`
- validar subida do backend

Etapa 4

- materializar testes de integração mínimos
- adaptar pipeline para PostgreSQL de teste

---

11. Fechamento

Este pacote converte o blueprint em um backend que já pode ser ligado com:

- autenticação JWT real
- RBAC inicial
- escopo inicial
- auditoria básica centralizada
- bootstrap sem stubs de aplicação
- testes mínimos para evitar regressões grosseiras

O próximo refinamento correto, depois desta materialização, é endurecer:

- refresh token com validade real e revogação completa
- checagem de escopo por domínio com dados reais
- testes de integração positivos com usuário autenticado
- CI rodando PostgreSQL real
- auditoria também em `UPDATE`, `DELETE lógico` e `EXPORT`